Properties of Task Environment

- **Fully Observable:** Agent can access **all relevant information** about the current state.
  Example: Chess
- **Partially Observable:** Agent can access **only some information**; some parts of the state are hidden/unknown.
  Example: Driving a car
- **Single Agent:** Only **one agent** is making decisions.
  Example: Sudoku
- **Multi-Agent: Multiple agents** interact, cooperate, or compete.
  Example: Chess
- **Deterministic:** An action always produces a **fixed, predictable result**.
  Example: 8-puzzle
- **Stochastic:** An action can produce **different outcomes due to uncertainty/randomness**.
  Example: Robot navigating slippery terrain
- **Episodic:** Each action/episode is **independent**; previous actions don't affect future episodes.
  Example: Image classification
- **Sequential:** Current actions **affect future states and decisions**.
  Example: Chess
- **Static:** Environment **does not change while the agent is deciding**.
  Example: Crossword puzzle
- **Dynamic:** Environment **can change while the agent is deciding**.
  Example: Self-driving car
- **Discrete:** States, actions, or time have **distinct/countable values**.
  Example: Chess
- **Continuous:** States, actions, or time can have **any value within a range**.
  Example: Robot movement
- **Known:** Agent **knows the rules/effects of actions** and how the environment works.
  Example: Chess
- **Unknown:** Agent **does not initially know the rules/effects** and must learn/discover them.
  Example: Robot exploring an unfamiliar environment

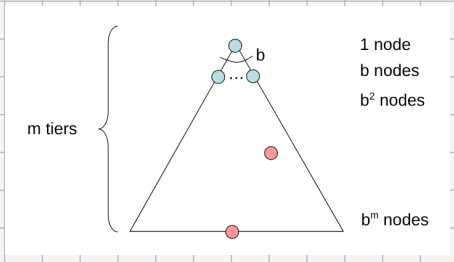
Quick memory

Observable → What can I see?
Agent → Who is acting?
Deterministic → Is the result predictable?
Episodic → Does the past matter?
Static → Does the world change?
Discrete → Countable or continuous?
Known → Do I know how it works?

Uninformed Search

BFS(queue):

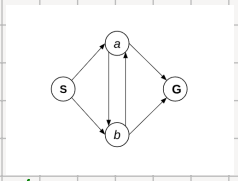
- Complete: yes
- Optimal: yes [if no weight in graph]
- Time: O(b^m)
- Space: O(b^m) [at a time, it stores the whole level]



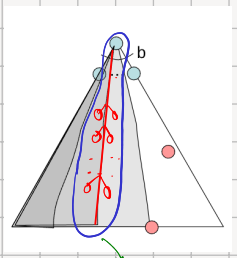
→ $O(b^0 + b^1 + b^2 + \cdots + b^m) = O(b^m)$

DFS(stack):

- Complete: yes [if tree is finite]
  no [if tree is infinite(loop)]
- Optimal: No
- Time: O(b^m)
- Space: O(bm)



↳ results in an infinite tree space (Loop)



DFS stores this much at a time in stack resulting in space complexity: O(bm)

Deth Limited Search[DLS](stack):

** DFS + (depth limit)

- Complete: No [goal may be at a deeper level than depth limit]
- Optimal: No
- Time: O(b^m) [only here, m=depth limit]
- Space: O(bm) ["]

Iterative Deepening Search(IDS):

** space advantage of DFS + shallow advantage of BFS

- Complete: Yes
- Optimal: Yes [if all weight=1]
  No [if different arc weight]
- Time: O(b^m)
- Space: O(bm)

Ex:

iteration = 0 → b^0

iteration = 1 → $b^0 + b^1$

iteration = 2 → $b^0 + b^1 + b^2$

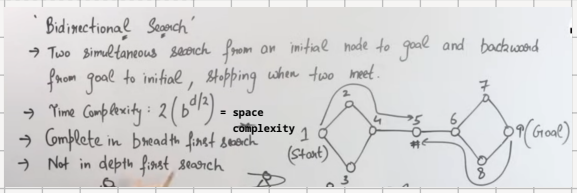
and so on . . .

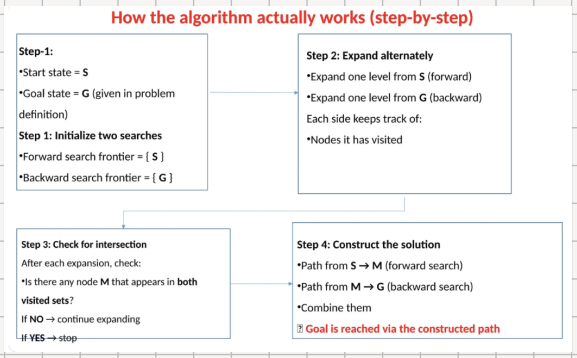
Uniform Cost Search (UCS)-> (Dijkstra):

** Cost level/effective depth → $\frac{C^*}{\varepsilon}$

- Time: $O(b^{C^*/\varepsilon})$
- Space: $O(b^{C^*/\varepsilon})$
- Complete: Yes
- Optimal: Yes

Bi-directional Search:



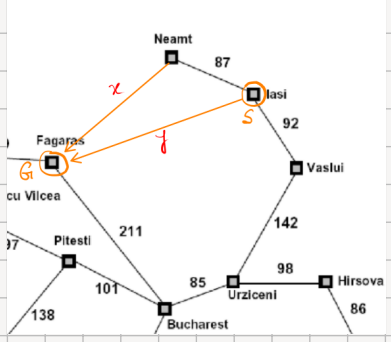


Informed Search

Greedy(Best First Search):

** f(n) = h(n); where h(n) is the heuristic cost from [n -> G] / forward cost.

- Complete: No.
- Optimal: No. [considers only heuristics]
- Time, Space: Good



if our heuristic is st. line distance, here [x < y], s=start, G=goal. So, Greedy returns the optimal path as [Iasi -> Neamt -> Fagaras] but there is no direct path between Neamt and Fagaras, hence it falls in an infinite loop

A* Search (UCS + Greedy):

** f(n) = g(n) + h(n);
- h(n) is the heuristic cost from [n -> G]/forward cost.
- g(n) is the cumulative cost from [S -> n]/backward cost

- Complete: Yes.
- Optimal: Yes. [Given admissible heuristics]

■ A heuristic *h* is *admissible* (optimistic) if:

$$0 \le h(n) \le h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal

if given multiple heuristics, take dominant, (bigger one, which is closer to the true cost)

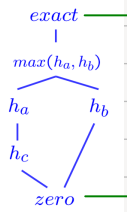
■ Dominance: $h_a \geq h_c$ if

$$\forall n : h_a(n) \geq h_c(n)$$

■ Heuristics form a semi-lattice:
- Max of admissible heuristics is admissible

$$h(n) = max(h_a(n), h_b(n))$$

■ Trivial heuristics
- Bottom of lattice is the zero heuristic (what does this give us?)
- Top of lattice is the exact heuristic



As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself.

heuristic=0; indicates its UCS. Doesnt have concern about heuristic, only concerns arc cost. Here, we expand all possible nodes to find the solution.

Memory Bounded Informed Search

IDA* (Iterative Deepening A*): [DFS]

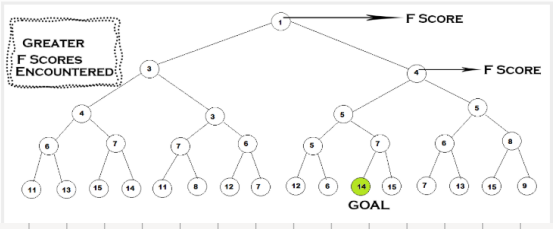
** Combines [DFS] with the heuristic evaluation of A*. Uses cost thresholds (f-values) to prune paths, running multiple depth-first iterations to save space at the cost of re-generating some nodes.

** Steps:
- Set threshold d = f(root).
  Run a depth-first search that only expands a node if [f(n) = g(n)+h(n) ≤ d]

- A node is goal-tested the moment it is generated.
  Threshold decides whether to expand.

- If the goal isn't found, set the next threshold to the smallest f-value that exceeded d and restart the whole search from the root.

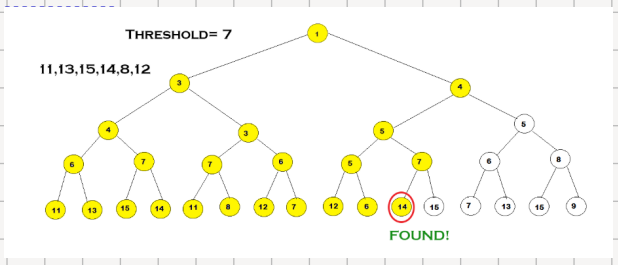
- Repeat until the goal is found (success) or no node was generated (failure).



Solution:

Iteration	Threshold	Cutoff nodes (f > threshold)	Next threshold
1	1	3, 4	3
2	3	4, 7, 6, 4	4
3	4	6, 7, 7, 6, 5, 5	5
4	5	6, 7, 12, 6, 7, 6, 8	6
5	6	7, 13, 7, 12, 7, 7, 13, 8	7
6	7	→ Goal found (14)	STOP

Solution Tree



SMA* (Simplified Memory Bounded A*): [A*]

** Operates like [A*] until available memory is completely full. When capacity is reached, it drops or prunes the worst node (highest f-cost) while backing up its value to parent nodes, balancing optimality and memory constraints.

** Steps:
- Set memory limit M and start at the root.

- Expand the leaf with the smallest [f(n)=g(n)+h(n)]

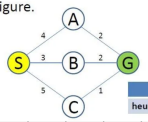
- A node is goal-tested the moment it is generated.

- If memory becomes full, remove the leaf with the largest f-value and back up its f-value to its parent.

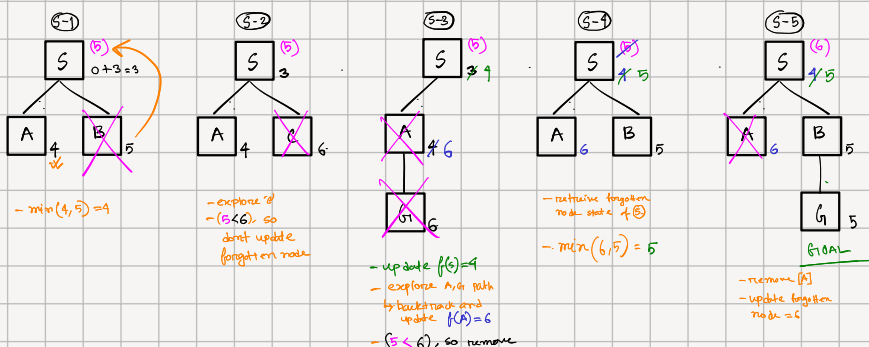
- Continue expanding the best leaf and deleting the worst leaf whenever memory is full.

- Repeat until the goal is found (success) or no nodes remain (failure).

• Perform SMA* (memory: 3 nodes) on the following figure.



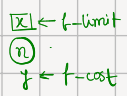
	S	A	B	C	G
heuristic	3	0	2	1	0



RBFS (Recursive Best First Search): [A*]

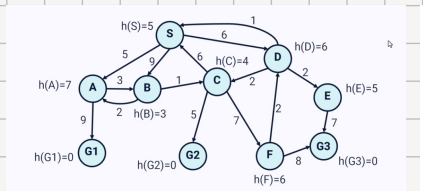
** Operates like [A*], with the difference that each node has internal memory, which it uses to identify a better path. When backtracking, the expanded subtree of that path is removed.

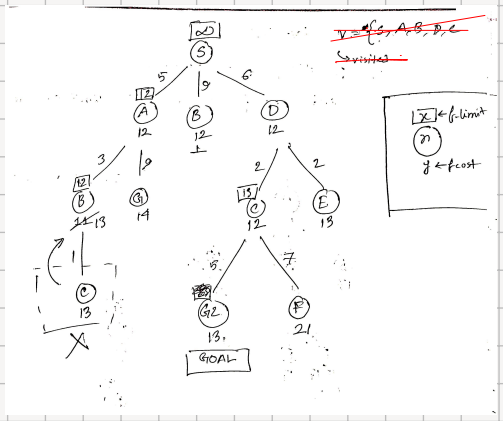
** Steps:



- Initially set start nodes f_limit = Infinity
    - f_limit denotes alternative best path if current path fails.
    - Current path fail if current [f_cost > f_limit]
    - If current path fails, backtrack and update parents f_cost for which the path failed, and remove that child.
    - Then start to explore other best options.

- f_cost = [g(n)+h(n)]

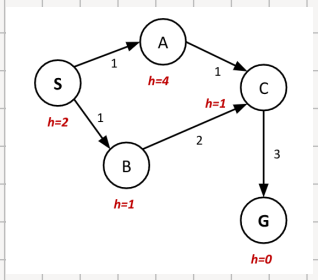


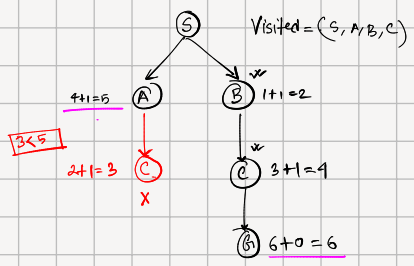


Graph Search

Uptil now, everything was 'Tree Search', lets learn Graph Search

Dont expand the same node twice, hence keep a set for visited node





here, [5<6] but we cant go that path because 'C' is already expanded.
** So only admissible heuristic isn't enought to find optimal solution in A* Graph Search.

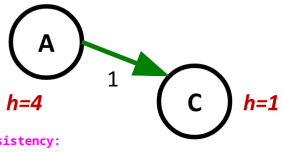
** For A* Graph Search, Heuristics must be [Admissible + Consistent], hence f(n) cost must be increasing as we do down.

■ Admissibility: heuristic cost ≤ actual cost to goal

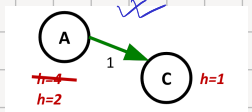
h(A) ≤ actual cost from A to G

■ Consistency: heuristic "arc" cost ≤ actual cost for each arc

h(A) – h(C) ≤ cost(A to C)



Consistency:
---
h(A)-h(C) <= actual cost
4 - 1 <= 1
3 <= 1 [inconsistent here]



2-1 ≤ 1
(consistent)

Local Search

```
Hill Climbing
-------------
**Moves to the first successor state that is better than
        the current state.

- Unlike the problems we've solved so far, this one doesn't
        store any path, hence doesn't return any shortest path,
        hence space complexity is good.

- Steps:
        - expand all successors of a parent
        - take min/max of the successors (according to problem)
        - compare with parent that whether the selected
          successor is better than parent
                - if better, remove parent & other succesors
                - if not, return parent as the solution

**Variations:
 - First-Choice HC: Generates successors and moves to the
      first one that is better than the current state.

 - Steepest Ascent HC: Evaluates all successors and
      moves to the best one.

 - Stochastic HC: Randomly chooses among the better
      neighbors and moves there.

 - Random-Restart HC: Repeatedly runs HC from different
      random initial states to escape local maxima.
```

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current ← MAKE-NODE(*problem*.INITIAL-STATE)
loop do
  neighbor ← a highest-valued successor of *current*
  if neighbor.VALUE ≤ current.VALUE **then return** *current*.STATE
  current ← *neighbor*

Figure 4.2 The hill-climbing search algorithm, which is the most basic local search technique. At each step the current node is replaced by the best neighbor; in this version, that means the neighbor with the highest VALUE, but if a heuristic cost estimate h is used, we would find the neighbor with the lowest h.

Simulated Annealing

- **Idea:** Inspired by the cooling process of metals; gradually reduces randomness to find a good global solution.
- **Main concept:** Unlike hill climbing, it can **accept a worse move** to escape local maxima/minima.
- **Temperature (T):** Controls how likely the algorithm is to accept a worse move.
  - High `T` → more likely to accept worse moves.
  - Low `T` → mostly accepts only better moves.
- **Acceptance probability:**

$$P = e^{-\Delta E/T}$$

  where `ΔE` = how much worse the new state is.
- **Cooling:** Temperature is gradually decreased according to a **cooling schedule**.
- **At T → 0:** Behaves more like hill climbing.

Basic Steps

1. Start with an initial state and high temperature `T`.
2. Generate a neighboring state.
3. If it is **better → accept it**.
4. If it is **worse → accept it with probability** $e^{-\Delta E/T}$.
5. Reduce `T`.
6. Repeat until `T` is very low / stopping condition is reached.

Key exam point: *Hill climbing never accepts a worse move; simulated annealing sometimes accepts worse moves to escape local optima.*

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  inputs: *problem*, a problem
    schedule, a mapping from time to "temperature"

current ← MAKE-NODE(*problem*.INITIAL-STATE)
for $t = 1$ **to** ∞ **do**
  $T \leftarrow schedule(t)$
  if $T = 0$ **then return** *current*
  next ← a randomly selected successor of *current*
  $\Delta E \leftarrow next$.VALUE $-$ *current*.VALUE
  if $\Delta E > 0$ **then** *current* ← *next*
  else *current* ← *next* only with probability $e^{\Delta E/T}$

Figure 4.5 The simulated annealing algorithm, a version of stochastic hill climbing where some downhill moves are allowed. Downhill moves are accepted readily early in the annealing schedule and then less often as time goes on. The *schedule* input determines the value of the temperature T as a function of time.

Adversarial Search

Minimax — Notes

1. Uses **DFS** to traverse the game tree.
2. **MAX node:** initialize `max = -∞`.
3. **MIN node:** initialize `min = +∞`.
4. Expand children until **terminal/depth-limit node**.
5. **MAX:** `max = max(max, child_value)`.
6. **MIN:** `min = min(min, child_value)`.
7. **Backtrack** and return the value to the parent.
8. At root, **MAX chooses the highest value**.

Complexity

- **Time:** `O(b^m)`
- **Space:** `O(bm)` — because DFS stores the current path and unexplored siblings.

`b` = branching factor, `m` = maximum search depth.

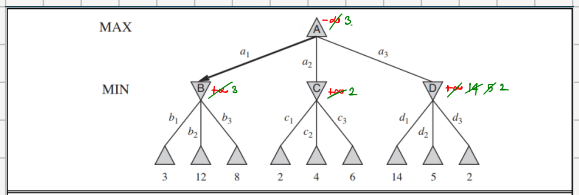


Figure 5.2 A two-ply game tree. The △ nodes are "MAX nodes," in which it is MAX's turn to move, and the ▽ nodes are "MIN nodes." The terminal nodes show the utility values for MAX; the other nodes are labeled with their minimax values. MAX's best move at the root is a_1, because it leads to the state with the highest minimax value, and MIN's best reply is b_1, because it leads to the state with the lowest minimax value.

```
Alpha-Beta Pruning
------------------
MAX → α (alpha) → initially -∞
MIN → β (beta) → initially +∞
Initially [α,β] = [-∞,+∞] and pass down the tree,
        later when [α,β] updates, send the updated values to lower level

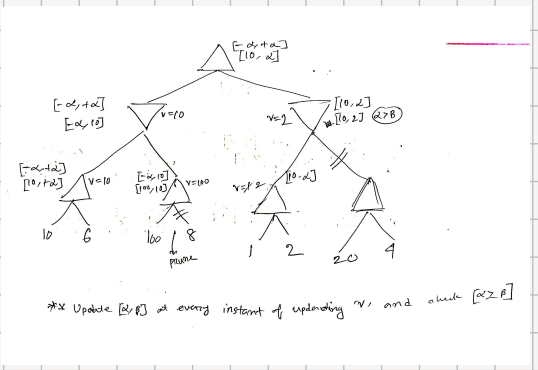
MAX node: (use v to update α)
 - v = max(v, successor)
 - α = max(α, v)

MIN node: (use v to update β)
 - v = min(v, successor)
 - β = min(β, v)

Pruning condition: [ α ≥ β ] -> prune remaining children

**Update [α,β] immediately after updating v at every instant,
        then check α ≥ β at every step.
```

Ex-1)



Ex-2)

